Tree-Based Methods

Regression, classification and ensembles

MATH 4432 Statistical Machine Learning · Tutorial T11

Department of Mathematics, HKUST

Fall 2026

Tree models: the task and the way trees are combined

Prediction task

Regression: a numerical response. Classification: a category.

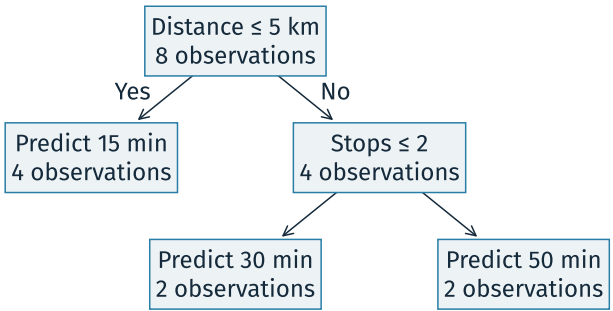
Model construction	What is fitted	How predictions are combined
Single tree	One partition of the inputs	One leaf gives the prediction
Bagging / random forest	Many varied trees	Average or vote
Boosting	A sequence of small models	Add each new contribution

CART = Classification and Regression Trees: a way to build individual trees.

Random forests add randomness to bagging; gradient boosting uses a loss to choose corrections.

A prediction follows a path through the tree

Constructed example: delivery time in minutes, using distance and stops.



New delivery

Distance = 8 km

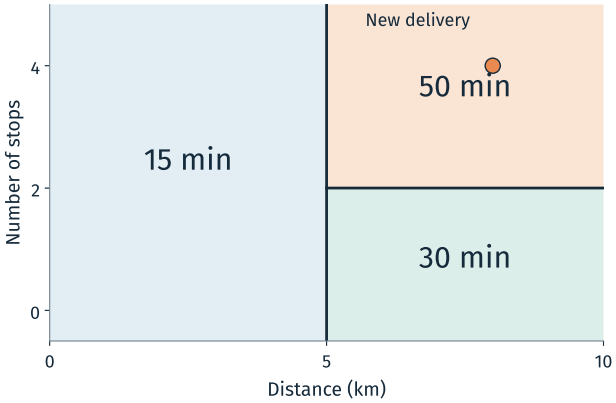
Stops = 4

Prediction: 50 min

Counts refer to training cases.
Leaf values are mean times.

Internal node: a question. Yes goes left, No goes right. A leaf gives the prediction.

The same tree partitions the predictor space



Three disjoint regions

Distance ≤ 5: 15 min

Distance > 5, stops ≤ 2: 30 min

Distance > 5, stops > 2: 50 min

Every input reaches one leaf. Within that region, the prediction is constant.

Prediction error: residuals, RSS and MSE

For n observations: y_i is the observed response and $\hat{y}_i$ is its prediction.

$$e_i = y_i - \widehat{y}_i$$

Residual: the signed prediction error

$$\text{RSS} = \sum_{i=1}^{n} e_i^2$$

Residual sum of squares: total squared error

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n} e_i^2 = \frac{\text{RSS}}{n}$$

Mean squared error: average squared error

Smaller is better on the cases being evaluated. Training error and validation error use different cases.

Why does a regression leaf predict the mean?

One leaf contains responses 2, 4 and 6. Use one constant c for all three.

$$S(c) = (2 - c)^2 + (4 - c)^2 + (6 - c)^2$$

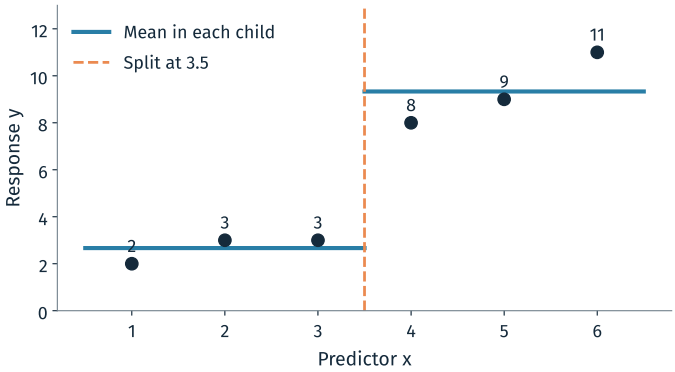
$$\frac{dS}{dc} = 2(c - 2) + 2(c - 4) + 2(c - 6) = 0$$

$$\hat{c} = 4 = \bar{y}_{\text{leaf}}$$

The mean minimizes squared error inside the leaf. No Gaussian assumption is needed.

A good split creates two better local predictions

Six training pairs. Try the rule $x \le 3.5$.



Before splitting

One prediction: 6

RSS = 72

After splitting

Left mean = 8/3

Right mean = 28/3

Total RSS = 16/3

Score both children using their fitted means. The split reduces total squared error.

Recursive binary splitting is a local search

At each node: try each predictor and each valid cutpoint.

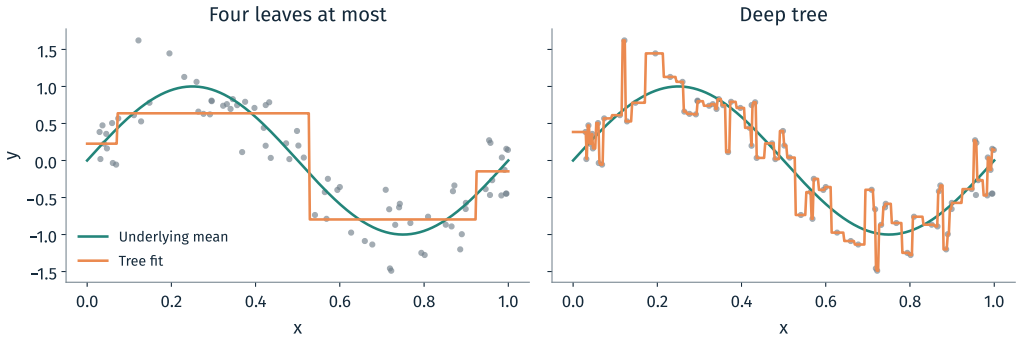
Rule in the six-point example	Left + right RSS
$x \le 2.5$	35.25
$x \le 3.5$	5.33 (best of all five cutpoints)
$x \le 4.5$	24.00

For numeric predictors, test midpoints between distinct observed values.
Choose the smallest combined RSS. Repeat within each child.

Greedy: the best next split need not belong to the best possible whole tree.

Growing deeper trades a simpler fit for more detail

The same 80 noisy observations, fitted at two depths.



Small tree: broad patterns, but some structure is missed.

Deep tree: closer training fit, but small leaves can follow noise.

Simulated data: $y = \sin(2\pi x) + \text{noise}$, noise SD = 0.32.

Pruning assesses whole branches

Stopping at the first small gain can be too short-sighted.

A weak first split may enable useful splits farther down.

Strategy	How complexity is controlled
Stop while growing	Limit depth or require enough observations per leaf
Grow, then prune	Compare smaller subtrees of a large fitted tree

Pruning can remove an entire branch. It cannot repair a split that was never grown.

Cost-complexity pruning charges for each leaf

$$C_\alpha(T) = \mathrm{RSS}(T) + \alpha\,|T|, \qquad \alpha \geq 0$$

|T| = number of leaves. Larger α makes extra leaves more expensive.

Candidate subtree	RSS	Leaves	Cost when α = 5
One leaf	72.00	1	77.00
Two leaves	5.33	2	15.33
Four leaves	0.50	4	20.50

Same six-point example: the two-leaf tree wins at α = 5.

The pruning path supplies candidates. Validation chooses the penalty.

Validation chooses how much to prune

Within each training fold	On its held-out fold
Grow a large tree from those training rows	Route held-out inputs through that tree
Prune it for each candidate α	Measure prediction error for each α

Average held-out error across folds and choose α.

Regrow on all development data, then prune at the chosen α.

Splits must be learned again within each fold. Held-out responses never choose a split.

Classification changes what a leaf predicts

The same splitting structure can predict a category.

Example leaf: 8 training cases, with 6 Yes and 2 No.

$$\widehat{p}(\text{Yes} \mid x \in R) = \frac{6}{8} = 0.75$$

Probability estimate: 0.75

Class prediction: Yes, the most common class

A small leaf can give an extreme, uncertain probability. Leaf size still matters.

Gini impurity measures how mixed a node is

In this node, p_k is the fraction of observations in class k.

With K classes, the fractions are nonnegative and add to 1.

$$G = \sum_{k=1}^{K} p_k(1 - p_k) = 1 - \sum_{k=1}^{K} p_k^2$$

Binary node	Class fractions	Gini impurity G
10 Yes, 0 No	1 and 0	0
8 Yes, 2 No	0.8 and 0.2	0.32
5 Yes, 5 No	0.5 and 0.5	0.50

A pure node has G = 0. Lower Gini means less mixing. Next, use it to compare splits.

Classification error and entropy are different scores

Use the same node fractions p_k, for k = 1, ..., K.

Classification error E

$$E = 1 - \max_k p_k$$

Fraction misclassified if every case gets the most common class.

Entropy H

$$H = -\sum_{k=1}^{K} p_k \log p_k$$

Another measure of class mixing.
Pure node: H = 0. Use natural logarithms.

Use 0 log 0 = 0. Gini and entropy can change even when the predicted class stays the same.

A split score combines child impurities, weighted by the numbers of cases in the children.

A split is scored using both child sizes

20 cases: 16 Yes and 4 No. Both children still predict Yes.

Node	Yes / No	Class error	Gini
Parent	16 / 4	0.20	0.32
Left child	9 / 1	0.10	0.18
Right child	7 / 3	0.30	0.42

$$G_{\text{after}} = \frac{10}{20}(0.18) + \frac{10}{20}(0.42) = 0.30$$

Weighted class error stays 0.20. Gini improves from 0.32 to 0.30.

Use Gini or entropy to grow. Validate with a metric suited to the prediction task.

A tree and a linear model represent different shapes

Let x contain p predictors. Consider a numerical response.

Additive linear model

$$f(x) = \beta_0 + \sum_{j=1}^{p} \beta_j x_j$$

One coefficient per predictor.
Its slope is the same across inputs.

Regression tree

$$f(x) = c_j \quad \text{when } x \in R_j$$

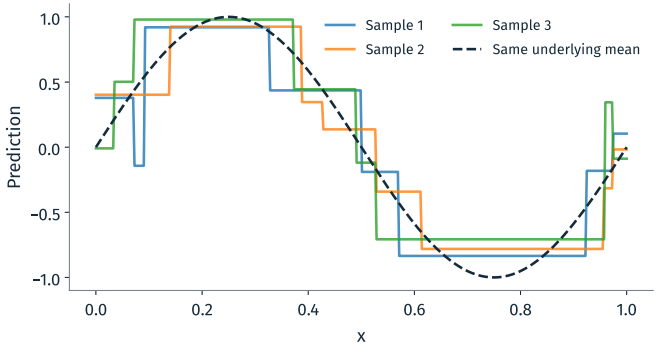
R_j is leaf region j; c_j is its prediction.
The effect of a variable can depend on the earlier splits in the path.

A tree learns thresholds and interactions. Within a leaf, its prediction stays constant.

Changing distance units from km to metres preserves the corresponding tree partition.

Small data changes can produce different trees

Here the inputs stay fixed. New response noise changes the fitted rules.



Instability motivates averaging several fitted trees, instead of trusting one set of splits.

Three simulated response samples, each with 80 points and the same underlying mean.

Bagging averages trees fitted to bootstrap samples

From n training rows, draw n with replacement. Repeat B times.

Bootstrap sample	Tree prediction at the same new x
Sample 1	12
Sample 2	18
Sample 3	15

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$$

Example average = 15

For classes: use majority vote, or average class probabilities and choose the largest.

Out-of-bag prediction excludes the case being checked

Training row IDs are 1, 2, 3, 4, 5, 6. Evaluate row 4, whose response is 9.

Tree	Bootstrap row IDs	Row 4 absent?	Use prediction
1	1, 1, 2, 3, 5, 6	Yes	6
2	1, 2, 4, 4, 5, 6	No	Exclude this tree
3	1, 2, 2, 3, 5, 5	Yes	8

OOB prediction for row 4 = (6 + 8)/2 = 7. Squared error = $(9 - 7)^2 = 4$.

Repeat for every row, using its own eligible trees. Average the errors.

For large n, about 36.8% of rows are absent from each sample, on average.

Averaging helps most when trees differ

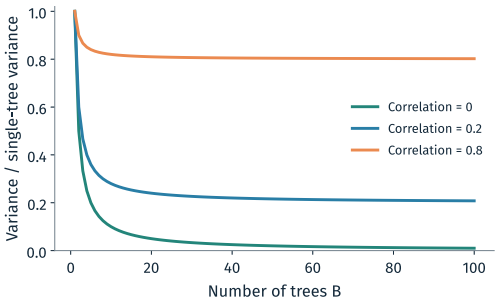
At a fixed x, suppose each tree has variance σ² and pairwise correlation ρ.

$$\mathrm{Var}(\bar{T}) = \sigma^2\left[\rho + \frac{1-\rho}{B}\right]$$

ρ = 0: variance falls as 1/B.

ρ = 1: averaging gives no reduction.

More trees cannot remove shared error.



Variance here includes repeated training samples and tree randomness.

Equal variances and equal pairwise correlations are a simplifying model. Derivation: B1.

Random forests randomize the candidate predictors

Bagging can keep choosing the same strong predictor near the top.

Random forest: at every node, draw m candidate predictors out of p.

Example: p = 5, m = 2	Allowed candidates	Chosen split
Root	x_2, x_4	Best cut on x_4
Next node	x_1, x_5	Best cut on x_1

Fresh candidates at each node encourage different trees. Splits still optimize fit locally.

m = p recovers bagging, when all other settings agree.

Random forest settings control different things

Setting	What changes	How to think about it
Number of trees B	Precision of the average	Enough trees for stable results
Candidates per split m	Similarity between trees	Smaller m can weaken each tree
Minimum leaf size / depth	Detail within each tree	Smaller leaves, finer patterns

Choose m and leaf size using OOB error or cross-validation.

At fixed settings, adding trees stabilizes an average. It does not add correction stages.

For classification, $m \approx \sqrt{p}$ is a common starting point. Validate the choice.

Boosting builds an additive model one step at a time

Averaging stabilizes predictions. Boosting adds corrections to the current fit.

Start with a simple model F_0. Build the model in stages:

$$F_m(x) = F_{m-1}(x) + a_m T_m(x)$$

T_m: the new tree. a_m: its weight. F_m: the combined model after m steps.

Keep the earlier trees fixed while learning the next contribution.

Averaging trees	Boosting trees
Each tree learns the original response	The next fit depends on the current model
Trees can be fitted separately	Trees are built in sequence

Boosting is the framework. Gradient boosting uses a loss function to choose the next contribution.

A loss defines what a correction should improve

A loss L(y, F) assigns a cost to prediction F when the observed response is y.

Smaller loss means a better prediction for that observation.

Half squared-error loss

$$L(y, F) = \frac{1}{2}(y - F)^2$$

Absolute-error loss

$$L(y, F) = |y - F|$$

Magnitude of the error	Half squared error	Absolute error
1	0.5	1
4	8	4

Squared loss gives large errors much more influence. The chosen loss determines the correction target.

Gradient boosting turns a loss into correction targets

The derivative with respect to F tells us how the loss changes as F increases.

Its negative is the local direction in which to change the prediction.

At round m, the correction target for observation i is:

$$r_{im} = -\frac{\partial L(y_i, F)}{\partial F}\Big|_{F = F_{m-1}(x_i)}$$

For the half squared-error loss just defined:

$$-\frac{\partial}{\partial F}\left[\frac{1}{2}(y - F)^2\right] = y - F$$

The correction target is the residual. Fit a small regression tree to these targets, then update.

Squared-error gradient boosting fits the residuals

Start with a constant prediction: $F_0(x)$ = the training mean of y.

At round m

$$r_{im} = y_i - F_{m-1}(x_i)$$

Fit a small regression tree h_m to the pairs (x_i, r_{im}).

$$F_m(x) = F_{m-1}(x) + \nu h_m(x), \qquad 0 < \nu \leq 1$$

v is the learning rate. Recompute residuals after every update.

One boosting step, with four observations

A stump has one split. Here $F_0 = 5$ and the learning rate is $v = 0.5$.

x	y	Initial fit	Residual	Stump h_1	Updated F_1
1	2	5	−3	−2	4
2	4	5	−1	−2	4
3	6	5	1	2	6
4	8	5	3	2	6

Best stump: split at $x = 2.5$. Its leaf values are the mean residuals.

Update = old fit + 0.5 × correction. Training RSS falls from 20 to 8.

The next tree sees new residuals

After the first update, residuals are −2, 0, 0, 2.

x	Current F_1	New residual	Stump h_2	Updated F_2
1	4	−2	−2	3
2	4	0	2/3	13/3
3	6	0	2/3	19/3
4	6	2	2/3	19/3

A best next split is x = 1.5. The right leaf averages 0, 0, 2.

Training RSS now falls to 4. This tree corrects the current fit, not the original mean.

There is a tied best split at x = 3.5. Either gives the same RSS reduction here.

Small correction trees build a more flexible function

Same training data and learning rate v = 0.1. Each tree has depth at most 2.



Orange: boosted fit. Green: underlying mean. Dots: noisy training responses.

The final prediction adds all corrections. It does not use only the last tree.

Classification needs a loss for predicted probabilities

For a binary response $y \in \{0, 1\}$, let p be the predicted probability of class 1.

Log-loss penalizes assigning a low probability to the observed class.

$$L(y, p) = -y \log p - (1 - y) \log(1 - p), \qquad 0 < p < 1$$

Observed class	Predicted probability p	Log-loss
y = 1	0.9	$-\log(0.9) \approx 0.105$
y = 1	0.1	$-\log(0.1) \approx 2.303$

Connection to entropy: average log-loss within a node, using its fitted class fractions, equals H.

Classification boosting changes a log-odds score

A real-valued score F represents probability p through the log-odds:

$$F = \log \frac{p}{1-p}, \qquad p = \frac{1}{1+e^{-F}}$$

F = 0 gives p = 0.5. Increasing F increases the probability of class 1.

$$L(y, F) = \log(1 + e^{F}) - yF, \qquad -\frac{\partial L}{\partial F} = y - p$$

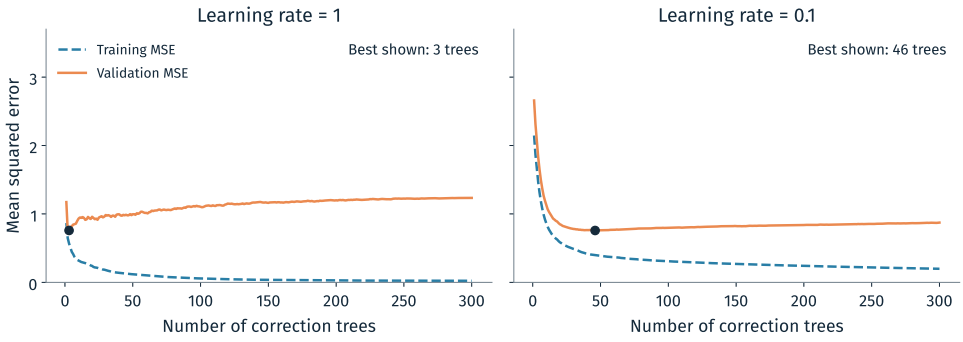
If y = 1 and p = 0.2: the target is +0.8, so raise the score.

If y = 0 and p = 0.8: the target is −0.8, so lower the score.

Regression trees learn numeric corrections, even for classification. Loss-matched leaf updates: C2-C3.

Learning rate and number of trees work together

One simulated training / validation split. All other settings are fixed.



A smaller learning rate usually needs more trees to make comparable progress.

Choose the stopping point using validation error. Training error alone keeps rewarding fit.

Illustration, not a universal ranking of learning rates. Simulation details: D1.

Tree depth controls the interactions boosting can learn

A stump has one split. Each correction depends on just one predictor.

$$F(x) = F_0 + g_1(x_1) + \cdots + g_p(x_p) \quad \text{(stumps)}$$

Many stumps can learn curved effects, but not interactions between predictors.

Depth 2: a second split can depend on the first split.

Example: the effect of stops differs for short and long deliveries.

Deeper trees allow richer interactions, but also more complex corrections.

Depth controls each correction. Learning rate and iteration count control accumulation.

Stochastic boosting uses a fresh subset each round

Optional extra randomness: use only a fraction q of the training rows.

At round m	Purpose
Compute current negative-gradient targets	Describe the remaining prediction error
Draw a fresh subset without replacement	For example, q = 0.5 of the rows
Fit the correction tree on that subset	Use leaf values appropriate to the loss
Add v times its correction to the model	The next round uses the updated model

The model still grows sequentially. Subsampling adds another tuning choice, alongside depth and v.

The methods differ in training target and combination

Method	What each tree learns	Final prediction
Single tree	Original response	One leaf prediction
Bagging	Original response on bootstrap rows	Average or vote
Random forest	Same, with random split candidates	Average or vote
Boosting framework	A contribution to the current fit	Weighted sum
Gradient boosting	Targets derived from the loss	Add scaled corrections

CART builds a tree. Boosting is a framework. Gradient boosted decision trees are often called GBDT.

A1 · Leaf means and the gain from splitting

For a fixed leaf R, compare c with the observed mean:

$$\sum_{i \in R} (y_i - c)^2 = \sum_{i \in R} (y_i - \bar{y}_R)^2 + n_R (c - \bar{y}_R)^2$$

The first term is fixed. The second is smallest at c = the leaf mean.

For a split into left and right children:

$$\mathrm{RSS}_{\text{parent}} - \mathrm{RSS}_{\text{children}} = \frac{n_L n_R}{n_L + n_R} (\bar{y}_L - \bar{y}_R)^2$$

A useful split separates child means and has enough observations on both sides.

The identity follows by expanding squared deviations around the two child means.

A2 · Why a weakest-link pruning threshold appears

A branch has L leaves. Replace it by one leaf at its root node t.

At a particular α, the two options tie:

$$\mathrm{RSS}(T_t) + \alpha L = \mathrm{RSS}(t) + \alpha$$

$$\alpha_{\mathrm{eff}}(t) = \frac{\mathrm{RSS}(t) - \mathrm{RSS}(T_t)}{L - 1}$$

Interpretation: extra RSS incurred per leaf removed.

Prune a branch with the smallest effective α, then recompute.
This yields a nested sequence of candidate subtrees.

Using MSE instead of RSS rescales α by the training sample size. Keep one convention.

A3 · What Gini and entropy measure

Within a fixed node, let p_k be the observed fraction in class k.

$$G = 1 - \sum_k p_k^2$$

Draw two labels independently from these fractions.

Gini is the probability that the labels differ.

$$-\sum_k n_k \log p_k = n_R \left(-\sum_k p_k \log p_k \right) = n_R H$$

Entropy is the fitted average negative log-likelihood in the node.

Hence child entropy is weighted by child size. Use 0 log 0 = 0.

B1 · Variance of an average of correlated trees

At fixed x, consider B random tree predictions.

Assume equal variances σ^2 and equal pairwise covariances $\rho\sigma^2$.

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B} T_b\right) = \frac{1}{B^2}\left[\sum_b \mathrm{Var}(T_b) + \sum_{b\neq c} \mathrm{Cov}(T_b, T_c)\right]$$

$$= \frac{B\sigma^2 + B(B-1)\rho\sigma^2}{B^2} = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

Bootstrap draws can be independent conditional on the observed dataset.
Across new training datasets, fitted trees still share the same data source.

The formula describes this unconditional variation under the stated equal-correlation model.

B2 · OOB probability and the error estimate

A bootstrap sample draws n times, independently, from n row IDs.

$$P(i \text{ absent}) = \left(1 - \tfrac{1}{n}\right)^n \longrightarrow e^{-1} \approx 0.368$$

Let B_i contain the trees whose bootstrap samples omit row i.

$$\hat{f}_{\text{OOB},\,i}(x_i) = \frac{1}{|B_i|} \sum_{b \in B_i} T_b(x_i)$$

$$\text{OOB MSE} = \frac{1}{n} \sum_{i=1}^{n} \left[y_i - \hat{f}_{\text{OOB},\,i}(x_i) \right]^2$$

Use enough trees so each row has OOB predictions. This is not literally leave-one-out CV.

C1 · Other regression losses give different targets

Write e = y − F for the residual. Choose a positive threshold δ.

Huber loss: squared near zero, linear for large errors

$$L_\delta(e) = \tfrac{1}{2}e^2 \quad \text{if } |e| \le \delta$$

$$L_\delta(e) = \delta|e| - \tfrac{1}{2}\delta^2 \quad \text{if } |e| > \delta$$

Negative-gradient target: e within the threshold, otherwise ±δ.

For absolute loss |e|, the target is sign(e), with a subgradient at zero.

After fitting the target pattern, choose leaf values to reduce the original loss. Huber limits extreme corrections.

C2 · Logistic leaf updates use positive curvature

Within a fixed leaf R, add the same score change γ to every case.

$$Q(\gamma) = \sum_{i \in R} \left[\log(1 + e^{F_i + \gamma}) - y_i(F_i + \gamma)\right]$$

$$Q'(0) = \sum_{i \in R} (p_i - y_i), \qquad Q''(0) = \sum_{i \in R} p_i(1 - p_i) > 0$$

$$\gamma_{\text{Newton}} = 0 - \frac{Q'(0)}{Q''(0)} = \frac{\sum_{i \in R} (y_i - p_i)}{\sum_{i \in R} p_i(1 - p_i)}$$

Use current probabilities, with $0 < p_i < 1$. Update scores by $\nu\gamma$.

Initialize $F_0 = \log(\bar{p} / (1 - \bar{p}))$, where $0 < \bar{p} < 1$ is the training class-1 fraction.

This is one local Newton step. A nearly pure leaf may need extra control of the step size.

C3 - A worked logistic leaf correction

The residual tree chooses regions. Then choose a score change for each leaf.

$$\gamma_R \approx \frac{\sum_{i \in R} (y_i - p_i)}{\sum_{i \in R} p_i(1 - p_i)}$$

This is one Newton step for logistic loss, not an exact minimizer in general.

Numerator: net error signal. Denominator: curvature of the loss.

Example leaf: outcomes 1, 1, 0. Current probabilities are all 0.5.

Numerator = 0.5. Denominator = 0.75. Correction $\gamma = 2/3$.

With $v = 0.1$: score 0 becomes 0.0667, so probability becomes 0.5167.

Learning rate v shrinks the score correction. Derivation and assumptions: C2.

C4 - Stagewise fitting and the lasso connection

Linear example: center y, standardize X and start from $\beta = 0$.

$$g_j = x_j^{\mathsf{T}}(y - X\beta), \qquad j^* = \arg\max_j |g_j|$$

$$\beta_{j^*} \leftarrow \beta_{j^*} + \varepsilon \operatorname{sign}(g_{j^*})$$

Select the predictor most aligned with the residual. Take a signed step of size ε.

Tree boosting chooses a small function to add, rather than one coefficient.

Small steps and early stopping can regularize the accumulated fit.

Connection, not identity: a general boosting path is not the lasso solution path.

D1 - Data and settings for the illustrations

All numeric examples and figures were constructed for this tutorial.

Illustration	Reproducible setup
Tree depth and instability	80 fixed x values from Uniform(0, 1). Mean $\sin(2\pi x)$. Gaussian noise SD 0.32. Seeds 4432 and 11, 12, 13.
Boosting curves	100 training and 300 validation cases. Seed 711. x from Uniform(0, 6). Independent Gaussian noise SD 0.8.
Boosting settings	Mean response $2\sin(1.5x) + 0.3x$. Squared-error loss. Depth 2, minimum leaf size 5, up to 300 trees.

The same training / validation split is used for both learning rates. Validation chooses M.

These examples illustrate mechanisms. They do not establish that one method always wins.

D2 · Where other tree-model names fit

These extensions use ideas already introduced in the main tutorial.

Name	Position in the family	Distinguishing idea
C4.5	Single classification trees	Can use multiway splits for categorical predictors
Extra Trees	Averaged tree ensembles	Randomize candidate cutpoints too
AdaBoost	A boosting method	Upweight misclassified cases
XGBoost / LightGBM / CatBoost	Gradient-boosted trees	Regularization, fast split search, and categorical-feature tools

Ask: how is one tree built, what does the next tree learn, and how are predictions combined?

Tree guide Ensemble guide XGBoost LightGBM CatBoost

References · Trees and aggregation

[1] James, Witten, Hastie and Tibshirani (2021)

An Introduction to Statistical Learning, 2nd edition

Chapter 8: trees, pruning, bagging, random forests and boosting.

[2] Hastie, Tibshirani and Friedman (2009)

The Elements of Statistical Learning, 2nd edition

Sections 9.2, 10, 15: tree models, additive fitting and random forests.

[3] Breiman (1996)

Bagging Predictors

Machine Learning 24, 123-140. Bootstrap aggregation and instability.

[4] Breiman (2001)

Random Forests

Machine Learning 45, 5-32. Random features, correlation and OOB prediction.

References · Boosting and tree methods

[5] Friedman (2001)

Greedy Function Approximation: A Gradient Boosting Machine

Annals of Statistics 29(5), 1189-1232.

[6] Friedman (2002)

Stochastic Gradient Boosting

Computational Statistics & Data Analysis 38(4), 367-378.

[7] Hastie, Taylor, Tibshirani and Walther (2007)

Forward Stagewise Regression and the Monotone Lasso

Electronic Journal of Statistics 1, 1-29. Conditions behind the lasso connection.

[8] scikit-learn documentation

Decision trees and gradient boosting

Further detail on split criteria and leaf updates.

Tree guide Ensemble guide